

* PCB *

Process Control Block.

- O.S creates PCB for every process that is to be executed. When the program is loaded into memory for execution, O.S creates PCB for it.

It includes key steps:-

1] Process Identification (PID):-

PCB contains a unique identifier for every process, it helps O.S to distinguish betⁿ the every other process.

2] Process state:-

PCB contains the current state of the process. It can be 'running', 'ready', 'terminated', 'waiting', etc. It helps O.S to know about what the process is doing for any given moment.

3] Process counter (PC):-

PCB stores the value of program counter, which indicates address of the next instructions that is to be executed after the process resumes execution.

4] Priority:-

Some operating system uses priority levels. It helps the O.S to know which process has higher importance.

5] Registers:-

All register inf is saved in PCB.

It helps O.S to save, retrieve the data when switching betⁿ processes.

6] Open files list:-

~~Pro~~ If the process has the files open for reading and writing, The open file list in PCB stores the inf about this files.

7] I/O information:-

PCB stores the input output status, and also the pending inp, out operations.

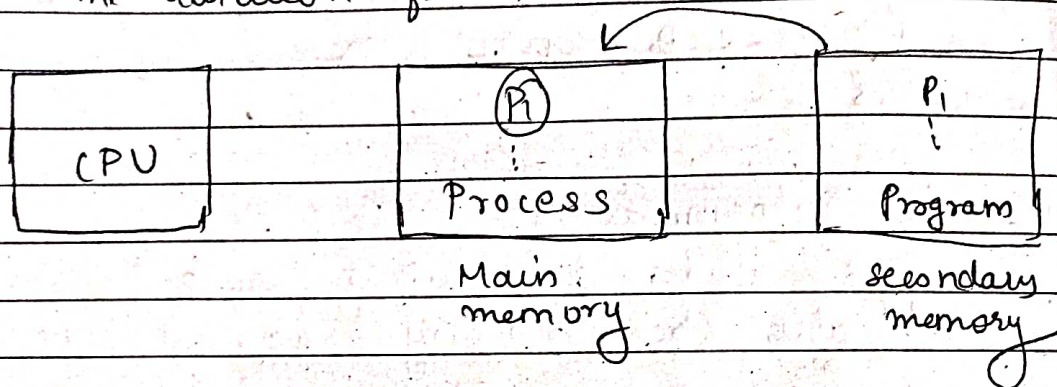
8] Protection:-

Process cannot interact with other process work place

Eg. Memory access.

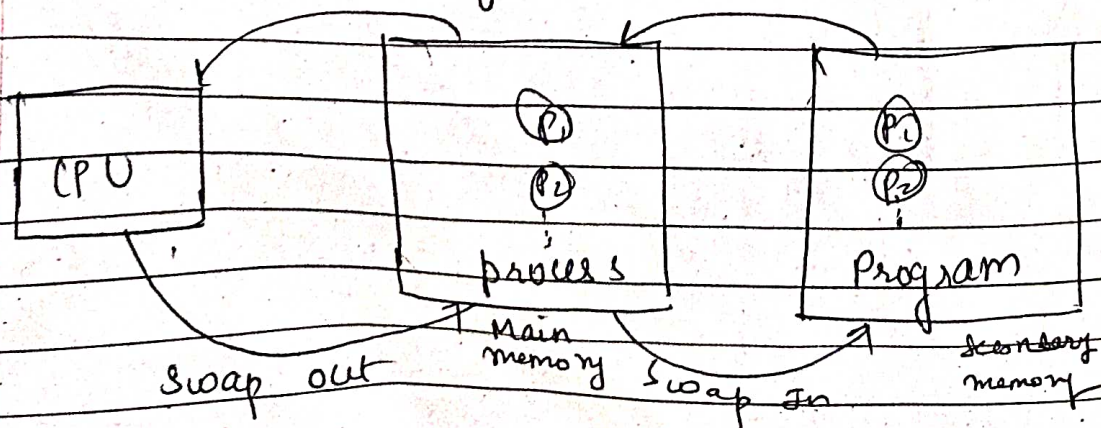
Long Term Scheduler (LTS)

- = job scheduler
- = selects the process from the queue & load it into memory.
- The freq of execution is from minutes to hour
- The duration for execution is more.



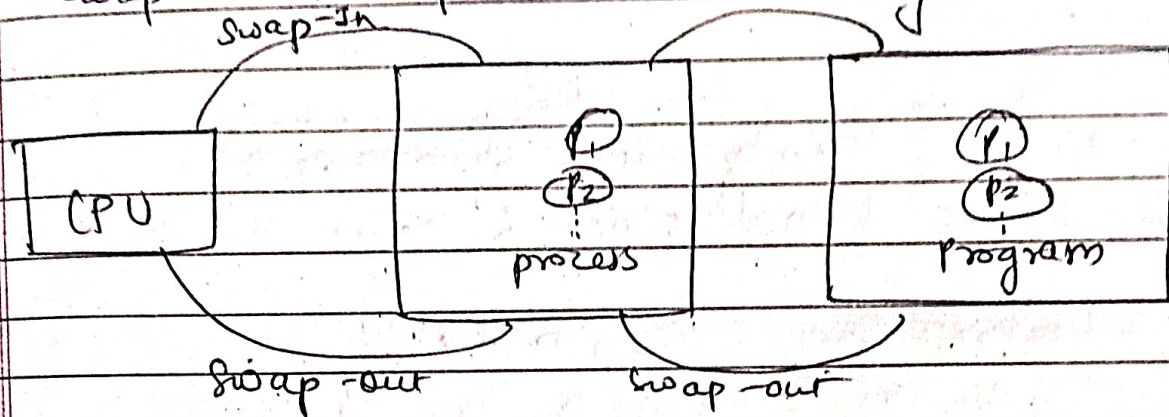
Short Term Scheduler (STS)

- CPU scheduler or Dispatcher.
- It selects the process among the processes that are ready to execute
- CPU scheduler makes the decision which process is going to execute next
- The freq of execution (sec to min)
(milliseconds to microseconds)
- The duration of execution is low.



MTS.

- some running process require i/p o/p operations.
- So, the process suspends from main memory & get placed on secondary memory, and after a while, it gets loaded in memory & continue where it left earlier.
- Swap-In & swap-out is done by MTS



* Context Switch

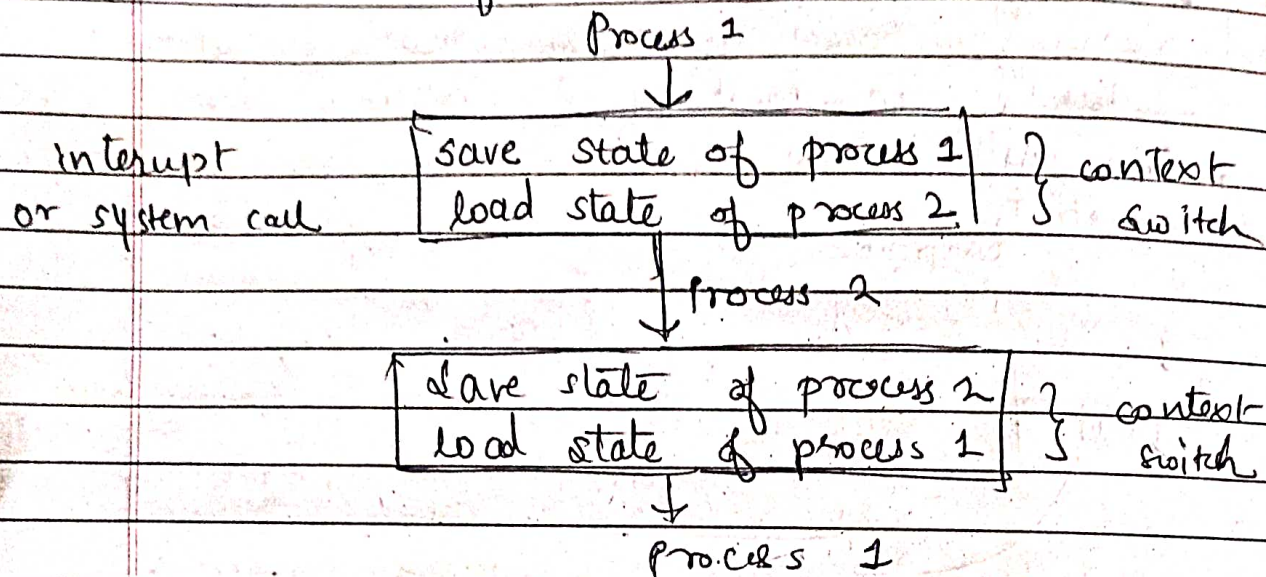
- When CPU wants to switch process from running state to ready or wait state, then CPU context get save in the PCB.
- It includes the following steps:-
 - 1] Save Current Context:-
The OS will save the current state of the process including, registers, program counters & relevant info.
 - 2] Select New Process / Context:-
The OS will select new process based on scheduling algorithms & priorities.

3] ~~Save~~ Load At New Context:-

O.S loads the new process from its PCB & starts executing it.

4] Resume Execution

The new process continues execution where it left earlier.



Time-sharing OS

- runs many tasks at once, switching betⁿ them quickly
- Gives quick responses to user actions (eg. typing)
- Ideal for interactive use, like modern computer systems.
- Prioritize users experience & responsiveness.

Multiprogramming OS

- runs many programs, but not necessarily at one time.
- Focuses on efficient use of CPU & resources.
- It is often used for batch processing (eg. data processing jobs).
- Aims to keep the CPU busy & maximize throughput.

Preemptive Scheduling	Non-Preemptive Scheduling
1] CPU is allocated to the processes for a limited time	1] CPU is allocated to processes till it terminates or switches.
2] Executing process is interrupted in middle of execution when high priority one comes.	2] Executing process is not interrupted in the middle of the execution
3] It is flexible	3] It is rigid.
4] Waiting Time is less	4] Waiting time is more
5] CPU utilization is high	5] CPU uti - low
6] cost associated	6] no cost associated

* Objectives of O.S :-

- To provide users a convenient interface to use comp system
- To make comp. system convenient to use in a efficient manner.
- To act as intermediary betⁿ user & hardware, to make easier for user to access & use other resources
- To hide details about hardware resources from the user.
- manage resources of comp system.

Q.

FCFS

- non-preemptive
- processes are executed in the order they arrive
- No process interruption
- simpler to implement
- May not be efficient.

RR

- preemptive
- processes are executed in circular queue fashion
- Process preempted after fixed time quantum
- More complex due to timeslicing
- More efficient due to CPU sharing.

- Adv & Disadv of writing O.S in C :-

Adv:-

- portability:-

Code can be run on diff hardware with minimal changes.

- maintainability:-

structured code makes it easier for long-term management.

- Developer-friendly:-

Many programmers know C.

- Efficiency:-

It is efficient, can handle hardware directly.

Disadv:-

- Not super-fast:-

- slower than very low-level language

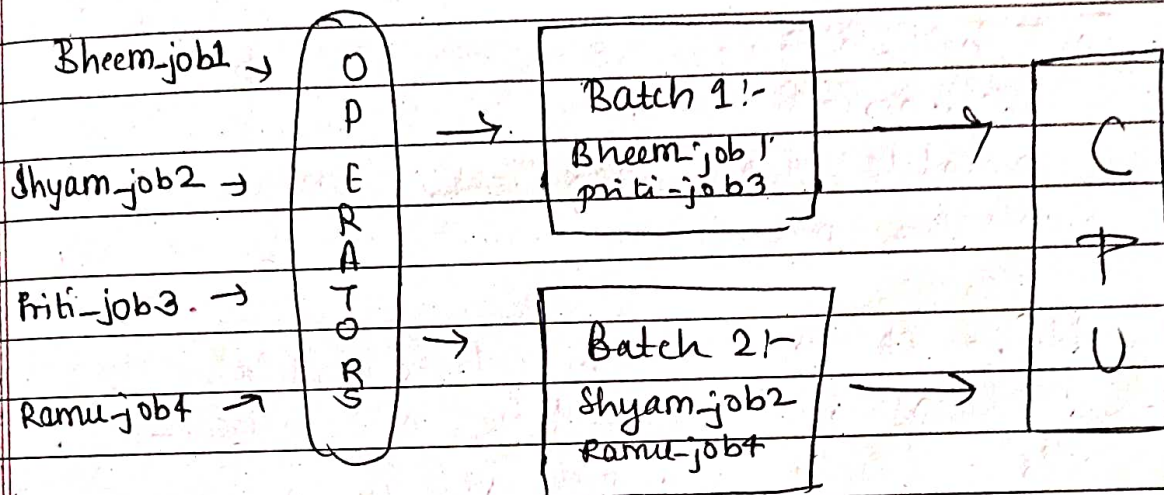
- Hardware limitation:-

may not use hardware to its fullest

• Complexity - ^{deep}
still requires ~~good~~ knowledge of OS.

* Types of O.S:-

1) Batch O.S:-

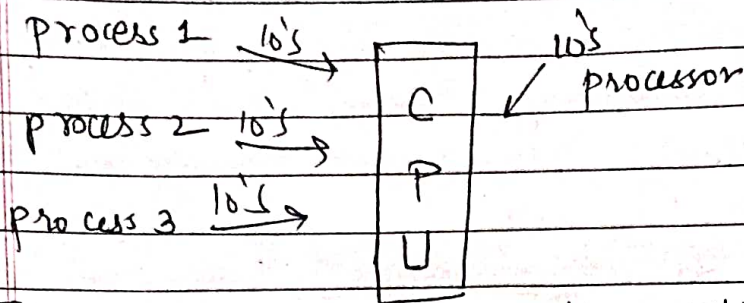


- Batch O.S works with similar kind of jobs together.
- In Batch O.S, user do not directly interact with comp system

Adv:- It is useful when we are working on a file which takes more time for its execution.

dis:- Once, the job is submitted, users cannot interact with it.

② Time-Sharing O.S:-

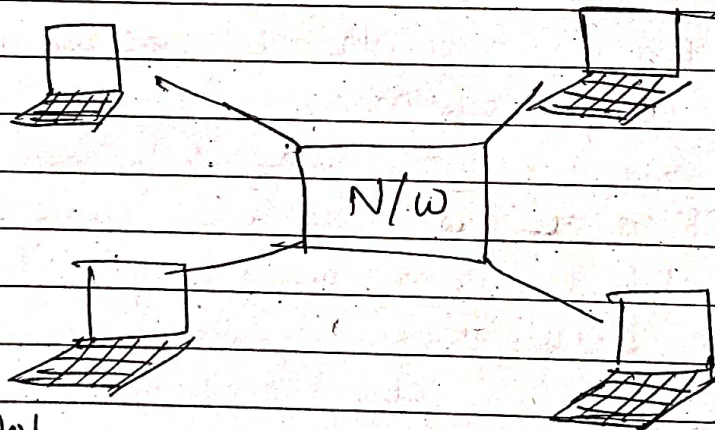


In Time sharing O.S, the CPU provides same time period for each & every task to complete its task as soon as possible whether process is short process or long process.

Adv:- If the process gets completed, then the other process time will increase.

Dis:- Data communication.

③ Distributed OS:-



- When many comp are interconnected with each other with network with purpose of sharing tasks.

Adv:- If the load is over used, then task is distributed among other comp.

Dis - If the network fails, the entire system work will stop.

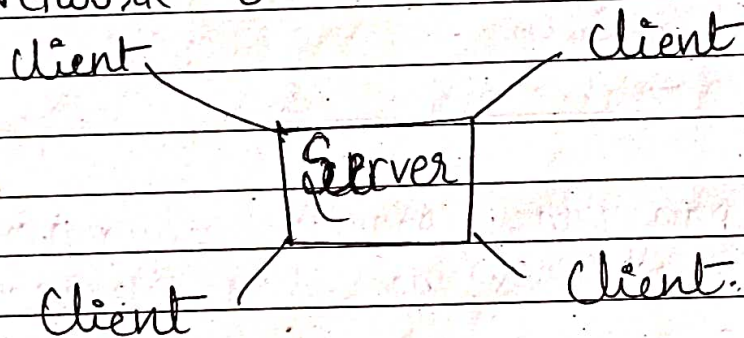
④ Real-Time OS:-

- Rt is useful when you want quick response
- for eg. Missile system
- Various org. like NASA, ISRO uses this O.S. for getting quick response

Adv:- Many org. using this O.S. for results

Dis :- Costly

⑤ Network OS:-

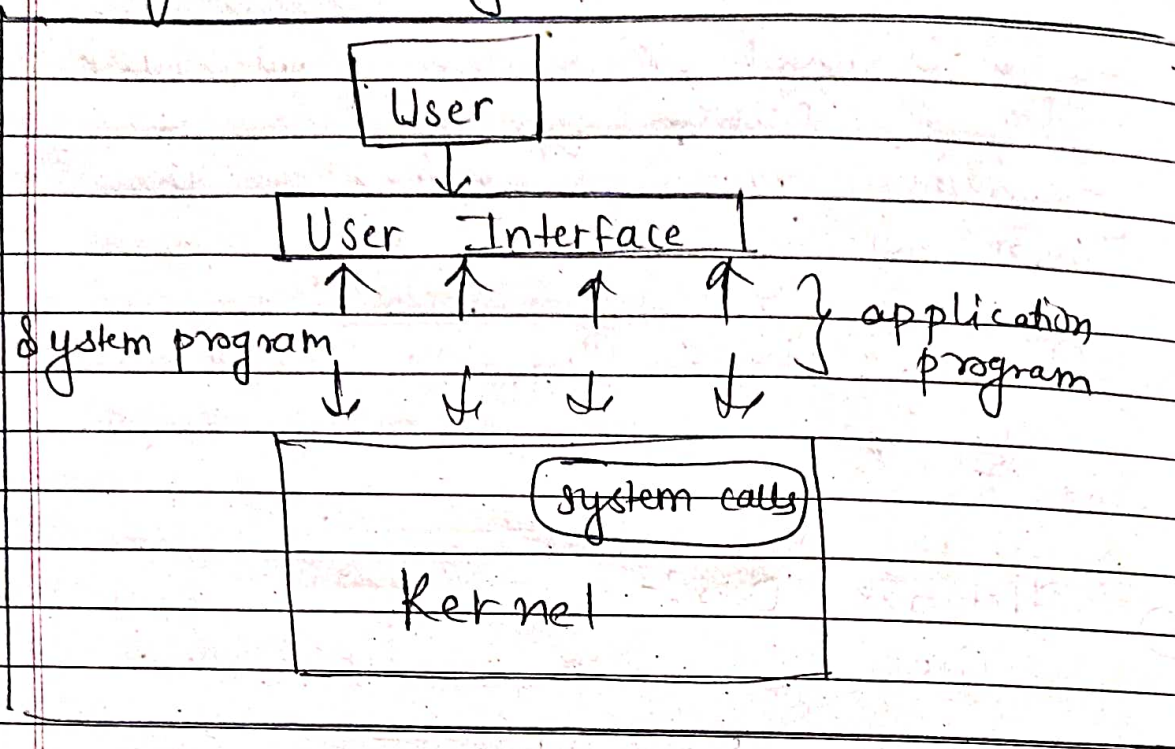


- N.O.S. has server that is connected to many other client's computer.
- If we want to share any files, or resources, it is useful.

Adv:- Security & we need to upgrade only server machine.

Dis :- If server crashes, entire system fails.

System Programs:-



- System programs provide environment to develop & execute the program.
- Types:-

1] Status Inf:-

System calls use status.

Eg. System date, time, available memory, how many users login, etc.

2] Communicatⁿ:-

Connection will be provided among processors & users.

Eg. Email, web browsers.

3] File Management:-

Creating file, deleting file, searching file, etc.

4] File modification:-

Data will be modified in the system file.
eg. Text Editor, command line.

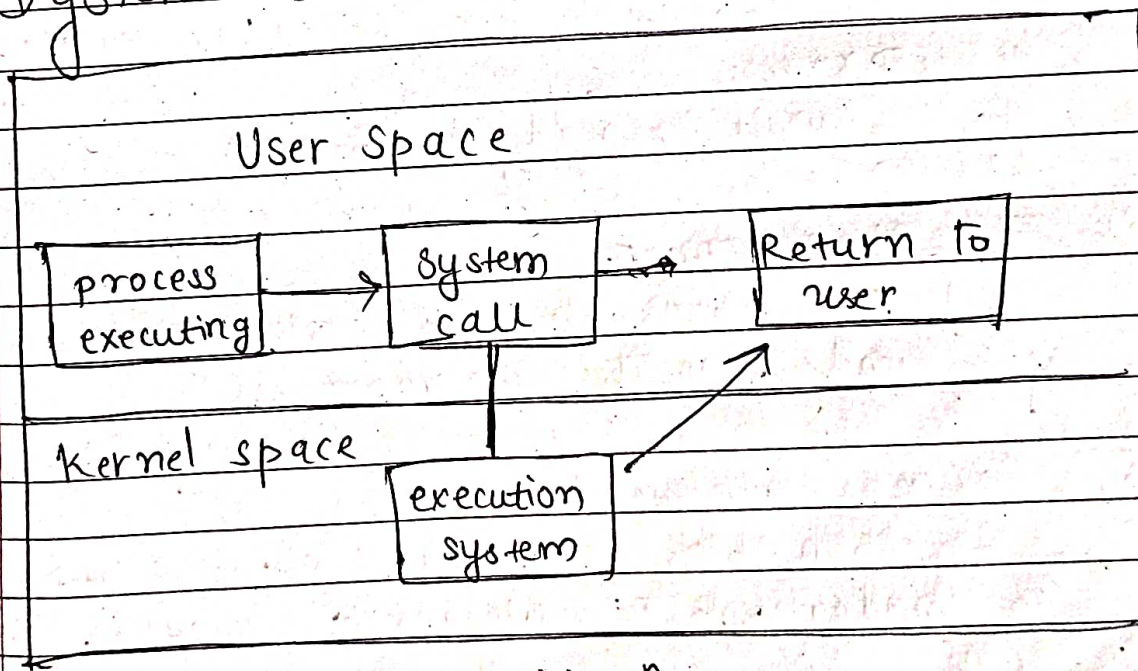
5] Program load^g & executing:-

Program will be loaded in the memory
& program will be executed.
eg. linkers, loaders.

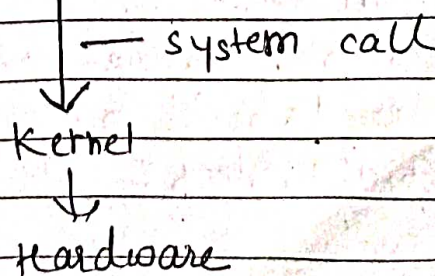
Application program

It provides services to users.
eg. Spreadsheets, Videos, Audio player.

* System Calls:-



User Application



- Sys. call is interface betⁿ user application and kernel
- It provides services to process via API (Application Programming Interface).
- When the process is executing and if it requires any resources, it will call 'interrupt'.
- Basically, Sys calls are entry point for kernel space.

Types:- [PDF-CI] :-

1] Process Control:-

It is responsible for creating, terminating the process.

Eg. wait(), ready(), fork()

2] File Management

It is responsible for file manipulations.
eg, read(), write(), readprocess().

3] Device Management:-

responsible for device manipulations:-
eg readconsole(), writeconsole(), read().

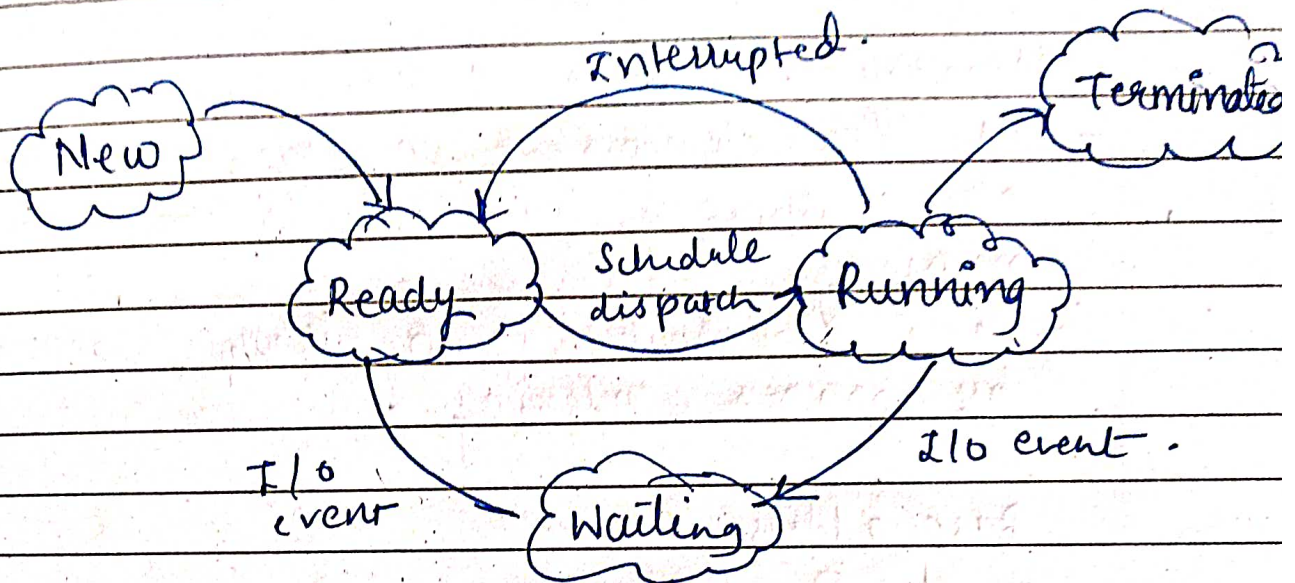
4] Inf. Maintenance:-

It is responsible for maintaining the inf. & inf shared betⁿ processes & O.S.
getpid(), sleep.

5) Communication:-

- It includes Inter process communication
eg `createpipe()`, `pipe()`.

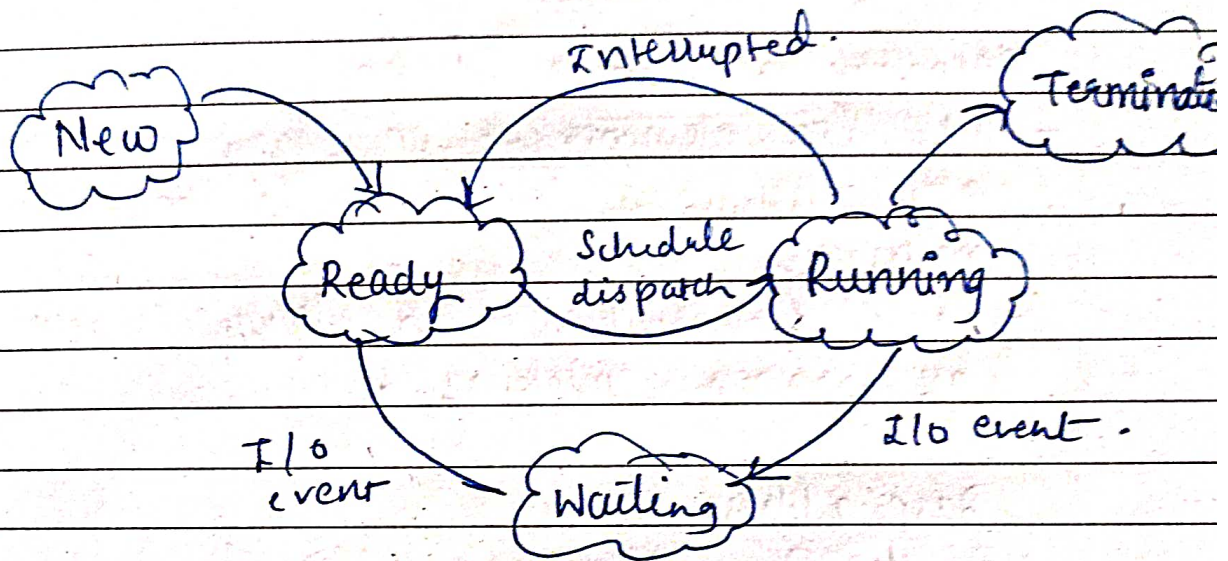
Process Life Cycle:-



5] Communication:-

- It include Inter process communication
eg `createpipe()`, `pipe()`.

Process Life Cycle:-



OS

3. Process Management & Synchronization

- * 1) Program under execution is called a process, it has various attributes like process ID, process state, process counter, priority, list of open files, etc.
- 2) It involves managing the execution of programs and tasks.
- 3) Process management ensures that each application runs efficiently without interfering with others.

* Process Synchronization:

- It is a mechanism that ensures multiple processes in a comp system work together without causing conflicts. It prevents data corruption & ensures the process executed in a coordinated manner.

Types → Cooperative processes

They willingly share resources & info. with each other, ensuring everyone is on same task.

1) Independent processes

Processes run independently. They do not communicate or coordinate with each other & carry out their own tasks.

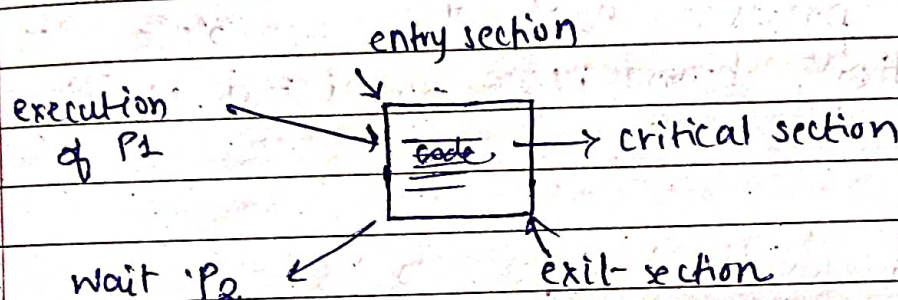
race condⁿ :- When two processes access a shared variable at same time race condⁿ occurs - Rainbow

PAGE: / /
DATE: / /

Critical Section Problem :-

- 1) A critical section refers to a segment of code that is executed by multiple processes, of which access shared resources.
- 2) These resources may be shared files, Memory, or other system resources that can be only accessed by one process at a time to avoid race conditions

3)



- 4) Once one process has entered the critical section, all other processes must wait until the executing process exits the critical section.
- 5) The purpose of synchronization is to ensure that only one process executes the critical section at time.

Code segments — ① Entry section
② Critical section
③ Exit section

6)

If the process satisfy three condition then there will be no race condition.

→

1) Mutual exclusion:-

At a time only one process is being executed in critical section

2) Progress:-

If one process does not require critical section that process should not stop other processes to get executed.

3) Bounded waiting:-

every process have to complete its execution before its predicted waiting time.

* Critical section solutions:-

Lock Variable:-

do

{

acquire lock

CS

release lock

}

① executes in user mode

② multiprocess so in

③ No mutual exclusion guarantee.

1

while (Lock == 1) {

Entry code

2

lock = 1

3

critical section

4

lock = 0

Exit code

Test-and-set Instruction

```
while (test-and-set (& lock));
```

```
CS
```

```
lock = false;
```

```
boolean Test-and-set (boolean *target)
```

```
{ boolean r = *target;
```

```
  *target = True;
```

```
  return r;
```

```
}
```

Semaphore:-

- It is an integer variable which is used in mutual exclusive way by the concurrent cooperative processes in order to achieve synchronization.

Counting
($-\infty$ to ∞)

Binary

(0,1)

- Implemented using wait(P) & signal(V).
- the wait operation decrements the value of the semaphore & signal operation increments the value of semaphore.



```

(1) wait(s)
{
    while (s <= 0 (s <= 0));
    s--;
}

```

```

(2) signal(s)
{
    s++;
}

```

Adv → Mutual exclusion

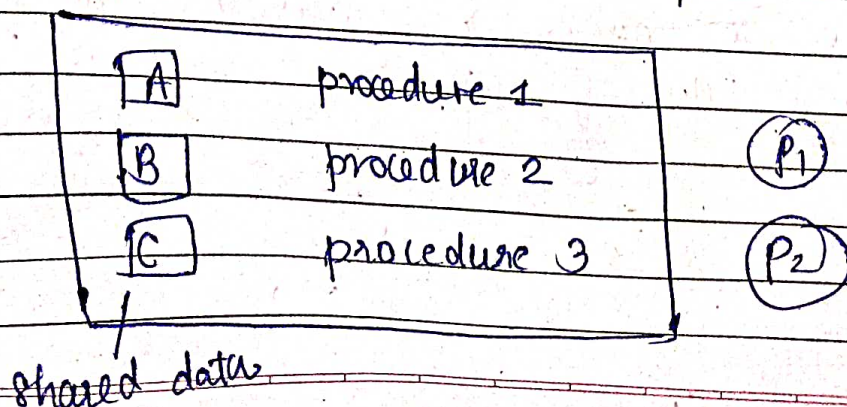
- efficient use of resources.
- prevents race condition

Disadv → high maintenance

- improper usage of semaphore lead to deadlock.
- Difficult to debug

Monitors:-

- ✓ It is a class, it contains shared data & procedure that operates on shared data.
- ✓ It contains conditional variables used to solve critical section problem.



Syntax of monitor:-

monitor

{

conditional variables;

variables;

procedure 1

{

==

}

procedure 2

{

==

}

}

Semaphore

Monitors.

1) It is a variable used to control access to a common resource in a concurrent system.

2) An integer variable

3) There is no condition variable concept.

4) When a process requires to access the semaphore, it performs wait() and performs signal() when releasing the resources.

5) Signal always has an effect-

1) It is a class that contains shared data & procedure that operates on shared data

2) An abstract datatype

3) It has condition variables.

4) It is a process uses procedures to access the shared variable in the monitor.

5) If a process wants to access data, it cannot access data directly. First we need to call procedure only one process will enter in a monitor & access data at a time.

6) No effect of signal C.

* ~~Producer consumer Problem / Bounded Buffer Problem~~

Reader's & Writers Problem:-

Readers only read the data they do not perform any updates. But writers can both read & write.

Problem:- It allow multiple readers to read at the same time

- Only one single writer can access shared data at same time.

dataset;

semaphore mutex=1

semaphore wrt = 1

integer read count = 0

// Structure of writer process:-

do

{

wait(wrt); $x=0$

// waiting is performed

signal(wrt); $\rightarrow 0+1$
 $= 1$

}

while (true);

// structure of reader process

do

{ wait(mutex); $x=0$

read count ++ $\rightarrow 0+1=1$

→

```

if (readcount == 1)
    wait(wrt);
    signal(mutex);
// reading is performed
    wait(mutex);
    readcount--;
if (readcount == 0);
    signal(wrt);
    signal(mutex);
} while (true);

```

Dining Philosopher Problem:-

- * Five philosopher are sitting around a table
- * The dining table has 5 chopsticks or bowl of rice in the middle.
- * Philosopher either eat or think
When philosopher want to eat, he use 2 chopsticks
- * when he wants to think, he keeps down both chopsticks
- * The problem :- Develop an algorithm where no philosopher starves. i.e. Every philosopher should eventually get a chance to eat.

chopstick [5] initialized to 1
structure of philosopher 1;
do

```

    wait (chopstick [i]);
    wait (chopstick [(i+1)%5]);
    // Eat;
    signal (chopstick [i]);
    signal (chopstick [(i+1)%5]);

```

Solution for Reader-Writer's Problem:-

It can be implemented by using demultiplexers.

semaphore is a integer variable in S, is accessed by only 2 atomic operations wait & signal

Code for Reader Process

wait (mutex);

```
if readcount == 1)
```

५

3

Signal mutex:

-- READ THE FILE?

wait mutex;

read count $\div -$

if read count == 0

f

Signal (write);

3

Signal (mutex);

Code for Writer

```
wait(write);
```

```
WRITE INTO THE FILE
```

```
signal(write);
```

Solution for Producer-Consumer

In producer-consumer problem, multiple producers items & place them in a shared buffer, while multiple consumers retrieve and consume these items from the buffer.

Monitor solⁿ is typically a lot easier to implement.

monitor

Code →

```
procedure producer():
```

```
    if count = SIZE:
```

```
        can_produce.wait;
```

```
    count++;
```

```
    can_consume.signal;
```

```
procedure consumer():
```

```
    if count = 0:
```

```
        can_consume.wait;
```

```
    count--;
```

```
    can_produce.signal;
```

```
}
```

2 vajta

Solution for Dining Philosopher

```
void philosopher
```

```
{
```

```
while (1)
```

```
{
```

```
wait (take - chopstick C[i]);
```

```
wait (take - chopstick ((i+1)%5));
```

```
...  
EATING NOODLE
```

```
...  
signal (put - chopstick C[i]);
```

```
signal (put - chopstick ((i+1)%5));
```

```
...  
THINKING
```

```
...  
}
```

```
}
```

→ first, wait operatⁿ is performed on take - chopstick C[i] & take chopstick ((i+1)%5). This shows philosopher i have picked up the chopsticks from its left and right. The eating funcⁿ is performed after that.

On completion of eating by philosopher i, signal op is performed, put & put. This shows that philosopher i have eaten & put down both left & right c.s. Finally, the philosopher starts think again.

Deadlock

Deadlock:-

If two or more processes are waiting for ~~some~~ happening of some event which never happens then we say that processes are involved in deadlock.

Necessary conditions for Deadlock:-

1) Mutual exclusion:- resource is ~~is~~ non-sharable.

Only one process can use resource at any given time. If another process requests the resource, the requesting process must wait until the resource is released.

2) Hold & wait:-

At least one process is holding one resource and waiting to acquire another additional resources that are currently held by another processes.

3) No-preemption:-

If one process has obtained a resource then the system cannot remove it from the process control until the process has finished using resource.

4) Circular wait:-

There is a cycle of processes and resources, such that each process in the cycle is waiting for a resource held by next process in the cycle.

Deadlock solutions:-

1) Deadlock prevention:-

To prevent a deadlock, os. must eliminate one of the four necessary conditions.

1) Mutual exclusion:-

ensure that the resources that cannot be shared is used in mutual exclusive manner.

2) Hold & wait:-

If a process is holding certain resource deny further request, it must release its original resource.

3) No preemption

Allow preemption of resources

4) Circular wait:-

It can be bypass if o.s. prevents the formation of a circle

2) Deadlock Avoidance:-

• Resource Allocation Graph:- (RAG)

Maintain a graph representing R.A. status.

Deadlock can be avoided by granting resource only if resulting graph does not contain a cycle.

• Banker's Algorithm:-

It calculates whether the granting a request will leave the system in a safe state or not.

3) Deadlock detection.

- RAG with Periodic checking
 - Periodically check the RAG for the presence of cycles. If a cycle is detected, a deadlock is assumed, & recovery measures are taken.

- Wait-Die & Wound-Wait schemes
These are techniques used in database systems to handle deadlock.

4) Deadlock recovery:-

- Process Termination:- (t.p)

Terminate one or more processes to break the deadlock. The t.p resources are released, allowing remaining process to continue.

- Roll-back

Rollback the state of one or more processes to a safe state.

- Preemption

Preempt resources from one or more processes to break deadlock.

Access Matrix

- It is a security model for of protection state in computer system.
- represented as a matrix.
- rows represent domain & column represents objects.
- Each entry in matrix consists of a set of access rights.

Diff types of rights:-

1) Read 2) Write 3) Execute 4) Print

eg.

	F ₁	F ₂	F ₃	Printer
D ₁	read		read	
D ₂				print
D ₃		read	execute	
D ₄	read write		read write	

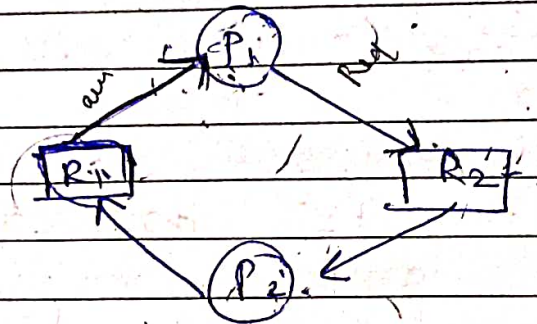
Here, there are 4 domains & 4 object -
three files & one printer.

- A process executing in D₁ can read files F₁ & F₃.
- A process executing in D₄ can read & write the files F₁ & F₃.
- A process executing in D₂ only prints.
- A process ex in D₃ reads file F₂ & execute F₃

* Wait for Graph algorithm.
with eg.

→ Deadlock - def.

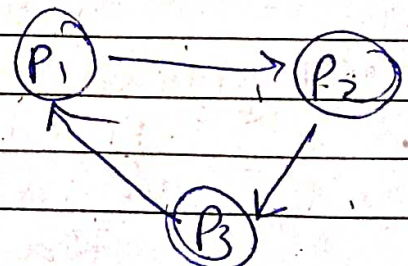
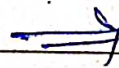
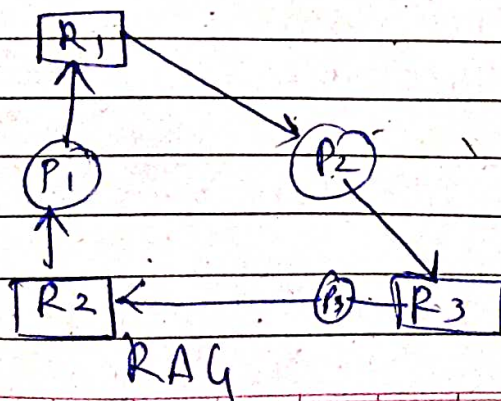
Eg. If there are three too processes P_1 and P_2 and P_1 is acquiring R_1 resource and R_2 is needed by P_2 which is acquiring R_2 & that is needed by P_1 .



Here, we found a cycle from P_1 to P_2 & again to P_1 . So, we say, that the system is in deadlock state.

Wait for Graph is a variant of RAG. In this algorithm, we have processes as vertices in the graph. If wait-for-graph contains a cycle, we say "deadlock".

for eg.



there's cycle
w-f-g.

✓ NO. 5

O.S.

* Memory hierarchy:-

* File:-

- A file is a named collection of related information that is recorded on secondary storage.
- Data files may be numeric, alphabetic, alphanumeric, binary.
- File is a sequence of bits, bytes, lines, or records.
- Many diff types of inf may be stored in a file - source or executable program, numeric or text data, photos, music, video & so on.

File attributes:-

A file's attributes may vary from one operating system to another.

1) Name:-

It is the only information kept in humanreadable form.

2) Identifier:-

This unique tag, usually a number, identifies the file within file system.

Type:-

This inf is needed for systems that support diff files

Location:-

It is the pointer to a device & to the location of file on that device.

Size:-

The current size of file (in bytes, words) & possibly max. size are included in this attribute.

Time date:-

This inf may be kept for creation, last modification, & last use. useful for protection & security.

File Operations:-

The O.S can provide system calls to create, write, read, reposition, delete and truncate files.

• Creating a file:-

Two steps necessary → space in file system
→ an entry for new file must be made in directory.

• Writing:-

we make system call specifying both the name of file & inf to be written

* Read a file:-

we use a system call that specifies name of the file & where the next block of file should be put

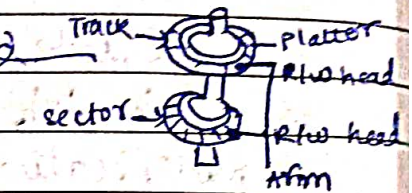
• Repositioning

The directory is searched for appropriate entry & current-file position pointer is repositioned to a given value.

• Deleting - to delete file from directory.

• Truncate - to erase all contents but keep its attributes.

* Disk Scheduling



• It is done by O.S. to schedule I/O req. arriving for the disk.

• It is also known as I/O scheduling.

• It is imp because:-

1) Multiple I/O req. may arrive by diff processes & only one I/O req. can be served at a time by the disk controller.

Thus, other I/O req. need to wait in waiting queue & need to be scheduled.

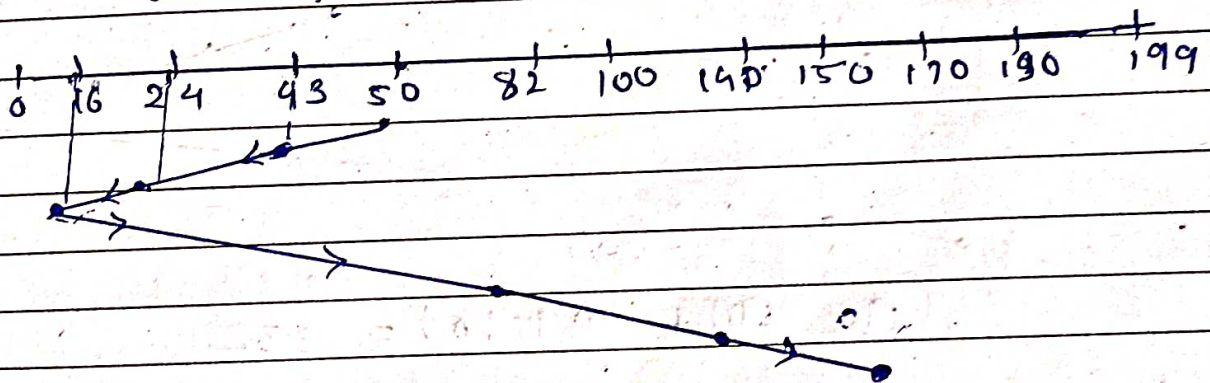
2) Two or more req. may be far from each other so can result in greater disk arm movement.

3) Hard drives are one of slowest part of comp. system & thus, need to be accessed efficiently.

There are disk scheduling Algorithms :-

- 1) SSTF (Shortest seek Time first) :-
- In SSTF, req. having shortest seek time are executed at first. So, the seek time of every req is calculated in advance in the queue & then they are scheduled acc. to calc'd seek time. As a result, the req. near the disk arm will be executed first.

Ex: SCAN Eg, order of req - (8, 2, 170, 43, 140, 24, 16, 190)
current pos'n head is $\rightarrow 50$



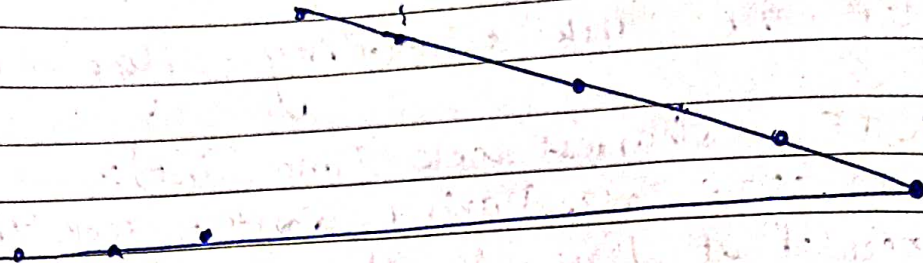
$\therefore$ total head movement :-

$$(50 - 43) + (43 - 24) + (82 - 16) + (140 - 82) = 208$$

- adv $\rightarrow$ avg response time dec
- disad $\rightarrow$ can cause starvation

2) look:-

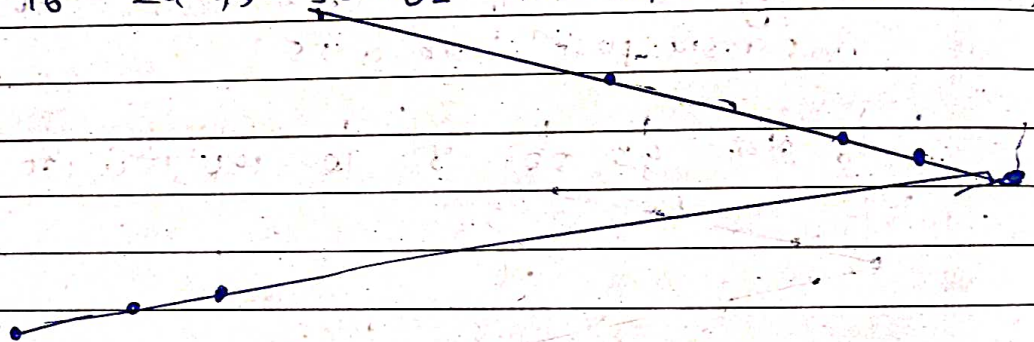
0 16 24 43 50 82 100 142 150 170 190 199



$$(190 - 50) + (199 - 46) = 314$$

3) Scan:- (we go till last)

0 16 24 43 50 82 100 142 150 170 190 199



$$(199 - 50) + (199 - 16) = 332$$

4) C-scan:-

* Thrashing :-

1) Thrashing occurs when a computer virtual memory resources are overused, leading to a constant state of paging & page fault.

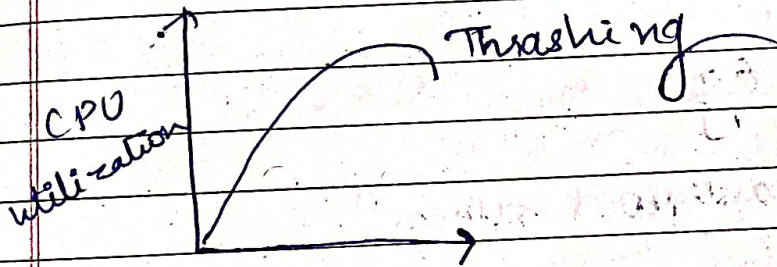
2) It causes the performance of the computer to collapse.

3) Because thrashing the CPU utilization is going to be reduce.

Algorithm during thrashing.

i) global page replacement algorithm.

ii) local page replacement algorithm.



* Allocation methods:-

The allocation methods define how the files are stored in the database disk blocks.

Three main file/disk allocation methods

1) contiguous allocation.

2) linked allocation.

3) Indexed allocation.

The idea behind these method is to provide

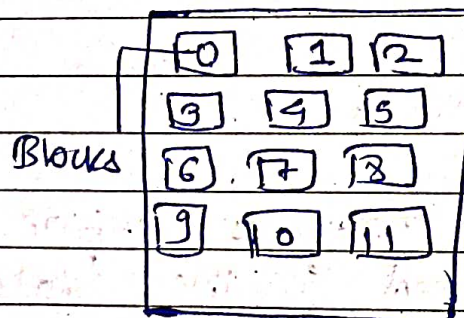
- Efficient disk space utilization
- Fast access to the file blocks.

1) Contiguous:-

In this, the block is allocated in such a manner that all the allocated blocks are in hand disk are adjacent. Assuming a file needs 'n' number of blocks & file begins with a block at position 'x', the next blocks to be assigned to it will be $x+1, x+2, x+3, \dots, x+n-1$ so that they are in a contiguous manner.

* Linked Eg:

contiguous allocation



Hard disk

File Name	start	length	Allocated blocks
file.txt	0	4	0, 1, 2, 3
sun.jpg	5	3	5, 6, 7
mov.mp4	9	3	9, 10, 11

Directory

Adv:- easy to implement
disk head movement is min

Disadv - • As it is pre-initialized, the size cannot increase.

- At time of creatⁿ, file size must be initialized.

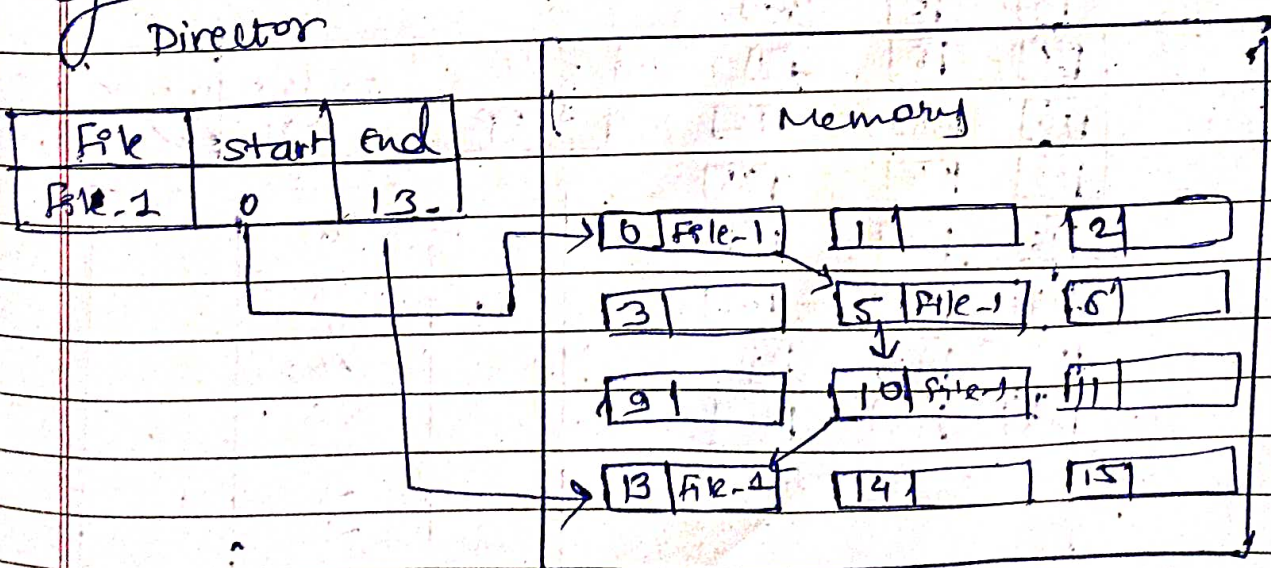
2) Linked subcatⁿ:-

It overcomes the drawbacks of the contiguous alkⁿ.

Here the file which we store on the hard disk is stored in a scattered manner accord^g to space available on hard disk.

So, as the name suggest linked fileⁿ, the pointers are used to point to the next block of same file.

Eg



Adv -

- no external fragmentation.
- memory is not needed in contiguous form

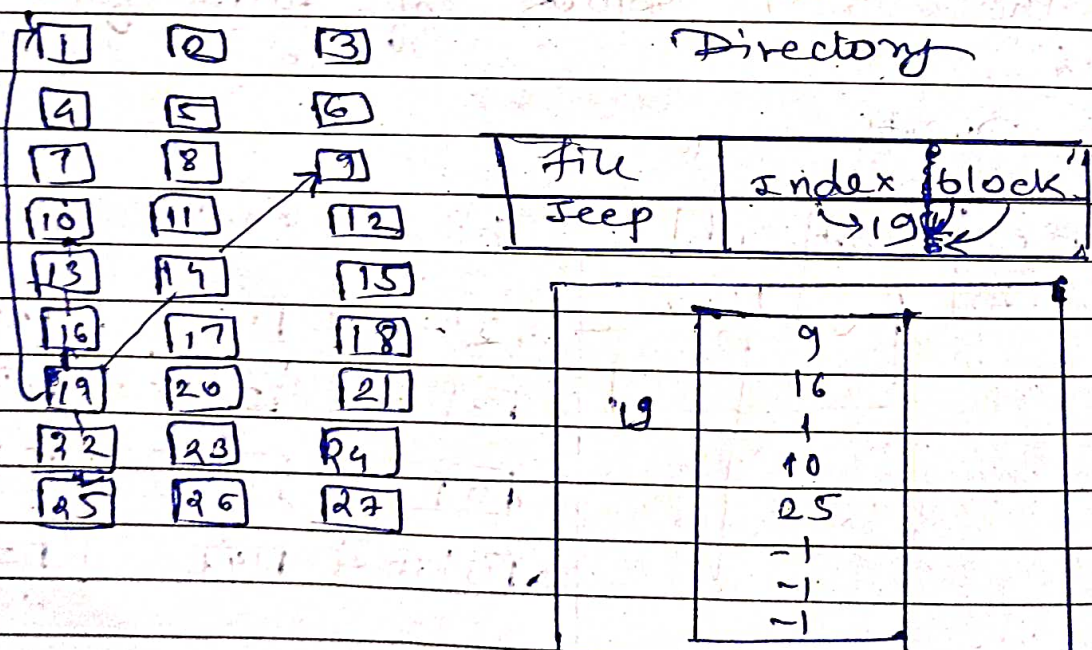
Disadv:-

- does not support random access
- if pointers r affected so disk blocks are also affected.

3} Indexed file Allocation -

It is somewhat similar to linked file allocation as indexed file also uses pointers but difference is here all the pointers are put together into one location called as "INDEX BLOCK".

That means, we will get all the locations of blocks in one index file.



-1 denotes empty index block list as file text is still so small to fill more blocks